

Sprint 5 Structured Notes: Nano and Git Setup

1. Current Progress

You are now in **Sprint 5** of the Relational Databases learning path.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance

So far, you have completed these foundation stages:

Sprint 1: Tools and VS Code

In Sprint 1, you learned how to prepare your coding environment.

You learned about:

- VS Code
- Code editors and IDEs
- Workspaces
- Project folders
- `index.html`
- Live Server
- Basic VS Code shortcuts
- Beginner extensions

Simple meaning:

Sprint 1 helped you prepare the place where you write and manage project files.

Sprint 2: Terminal, Shell, and Command Line Basics

In Sprint 2, you learned what happens when you type commands.

You learned about:

- Terminal
- Command line
- Shell
- Bash
- Command history
- Auto-complete
- Clearing the terminal screen

- Stopping a running command

Simple meaning:

Sprint 2 helped you understand the environment where commands are typed and run.

Sprint 3: Bash File and Folder Commands

In Sprint 3, you learned how to use Bash commands to work with files and folders.

You learned commands such as:

- pwd
- cd
- ls
- mkdir
- touch
- echo
- cat
- less
- mv
- cp
- rm
- >
- >>

Simple meaning:

Sprint 3 taught you how to move around folders, create files, read files, rename files, copy files, delete files, and write text into files using commands.

Sprint 4: Bash Scripting Basics

In Sprint 4, you learned how to save Bash commands inside a file and run them together.

You learned about:

- Bash scripts
- .sh files
- Shebang
- Interpreter paths
- Variables
- Variable interpolation
- Arrays
- for loops
- do and done
- echo logging

- `rsync` idea
- Binaries

Simple meaning:

Sprint 4 taught you how to turn Bash commands into a reusable script.

Sprint 5: Nano and Git Setup

Now you are starting **Sprint 5**.

Sprint 5 is about two important things:

1. Editing files directly inside the terminal using **Nano**.
2. Setting up **Git** so you can start tracking project changes.

Simple meaning:

Sprint 5 prepares you to use Git correctly before creating repositories and commits in Sprint 6.

2. Main Goal of Sprint 5

The main goal of Sprint 5 is:

Be ready to edit files in the terminal and configure Git correctly.

By the end of Sprint 5, you should be able to:

- Open a file using Nano.
- Type text inside Nano.
- Save a file in Nano.
- Exit Nano.
- Understand basic Nano shortcut symbols.
- Explain what Git is.
- Explain what version control means.
- Explain what a commit is.
- Explain what a branch is.
- Understand the difference between Git and GitHub.
- Check whether Git is installed.
- Install Git if needed.
- Set your Git username.
- Set your Git email.
- Set VS Code as your Git editor.
- View your Git configuration.

Simple meaning:

Sprint 5 is not about making commits deeply yet. It is mainly about preparing Git correctly.

3. Why Sprint 5 Is Important

Sprint 5 is important because Git is one of the most important tools used by developers.

Later, you will use Git when working with:

- SQL files
- PostgreSQL project folders
- README files
- Bash scripts
- Backend projects
- Database projects
- GitHub repositories
- Team collaboration
- Pull requests
- Real software projects

Before you can use Git properly, you must install it and configure your identity.

Simple meaning:

Git needs to know who you are before it can correctly save your project history.

Sprint 5 is also important because sometimes you may need to edit files directly in the terminal. Nano helps with that.

Simple meaning:

Nano helps you edit files when you are working inside a terminal.

Part 1: Nano Editor

4. What Is Nano?

Nano is a text editor that works inside the terminal.

Usually, you may write files using VS Code. But sometimes, especially when using a server or terminal-only environment, you may need to edit a file directly from the terminal.

That is where Nano helps.

Simple meaning:

Nano is a beginner-friendly text editor inside the terminal.

Example command:

```
nano notes.txt
```

This opens a file called `notes.txt` in Nano.

If the file already exists, Nano opens it.

If the file does not exist, Nano can create it when you save.

5. Why Learn Nano?

You should learn Nano because it is useful when:

- You are working in a terminal.
- You are connected to a server.
- You need to quickly edit a small file.
- You do not want to open a full code editor.
- You are fixing a configuration file.
- Git opens a terminal editor.

Simple meaning:

Nano is useful for quick file editing inside the terminal.

6. Nano Compared with Vim and Emacs

There are other terminal editors, such as:

- Vim
- Emacs

Vim and Emacs are powerful, but they can be difficult for beginners.

Nano is simpler because it shows common shortcuts at the bottom of the screen.

Simple meaning:

Nano is easier for beginners than Vim and Emacs.

7. Opening a File with Nano

To open a file with Nano, use:

```
nano filename
```

Example:

```
nano sprint5.txt
```

This opens a file called `sprint5.txt`.

Simple meaning:

```
nano sprint5.txt
```

 opens the file in the terminal editor.

If the file does not exist yet, you can type content and save it. Nano will create the file.

8. Typing Text in Nano

After opening a file with Nano, you can type text normally.

Example text:

```
I am learning Sprint 5.  
Nano helps me edit files in the terminal.
```

Simple meaning:

Nano lets you type and edit text inside the terminal.

9. Moving the Cursor in Nano

In Nano, you can move the cursor using the arrow keys:

- Up arrow

- Down arrow
- Left arrow
- Right arrow

Simple meaning:

Use arrow keys to move around the text in Nano.

10. Understanding Nano Shortcut Symbols

At the bottom of Nano, you will see shortcuts.

Nano uses symbols such as:

^

and:

M-

These symbols have special meanings.

11. What Does Mean in Nano?

In Nano, the symbol:

^

means **Ctrl**.

Example:

^O

means:

Ctrl + O

Another example:

^X

means:

Ctrl + X

Simple meaning:

In Nano,  means press Ctrl.

12. What Does Mean in Nano?

In Nano, the symbol:

M-

means the **Meta key**.

On many keyboards, the Meta key is usually:

Alt

Example:

M-U

usually means:

Alt + U

Simple meaning:

In Nano,  usually means press Alt.

13. Saving a File in Nano

To save a file in Nano, press:

Ctrl + O

Nano calls this:

Write Out

After pressing `Ctrl + O`, Nano asks you to confirm the file name.

Press:

Enter

Simple meaning:

Ctrl + O saves the file. Then Enter confirms the file name.

14. Exiting Nano

To exit Nano, press:

Ctrl + X

Simple meaning:

Ctrl + X closes Nano.

If you made changes but did not save, Nano may ask whether you want to save before exiting.

15. Basic Nano Practice

Step 1: Open a file

nano sprint5.txt

Step 2: Type this text

```
Sprint 5 practice  
I am learning Nano.
```

Step 3: Save the file

Press:

```
Ctrl + O
```

Then press:

```
Enter
```

Step 4: Exit Nano

Press:

```
Ctrl + X
```

Step 5: Check the file content

```
cat sprint5.txt
```

Expected output:

```
Sprint 5 practice  
I am learning Nano.
```

Simple meaning:

This practice proves that you can create, edit, save, exit, and read a file.

16. Important Nano Shortcuts

Nano Shortcut	Meaning	Simple Explanation
<code>^G</code>	Help	Opens help information
<code>^O</code>	Write Out	Saves the file
<code>^F</code>	Where Is	Searches for text
<code>^K</code>	Cut	Cuts the current line or selected text
<code>^T</code>	Execute	Runs a command or tool from Nano
<code>^C</code>	Location	Shows cursor location
<code>M-U</code>	Undo	Undoes the last change
<code>M-A</code>	Set Mark	Starts selecting text
<code>M-J</code>	To Bracket	Moves to matching bracket
<code>M-B</code>	Previous	Moves to previous match or item
<code>^X</code>	Exit	Exits Nano
<code>^R</code>	Read File	Inserts another file into the current file
<code>^\ </code>	Replace	Finds and replaces text
<code>^U</code>	Paste	Pastes cut/copied text
<code>^J</code>	Justify	Formats paragraph text
<code>^/ </code>	Go To Line	Moves to a specific line number
<code>M-E</code>	Redo	Redoes an undone change
<code>M-6</code>	Copy	Copies selected text or current line
<code>^B</code>	Where Was	Moves backward or searches previous position
<code>M-F</code>	Next	Moves to next match or item

Beginner note:

You do not need to memorize all Nano shortcuts now.

For Sprint 5, focus mainly on:

```
Ctrl + O = Save  
Ctrl + X = Exit
```

Simple meaning:

For beginners, saving and exiting are the most important Nano skills.

Part 2: Git Setup

17. What Is Git?

Git is a version control system.

Version control means it tracks and manages changes in a project.

Simple meaning:

Git saves the history of your project.

Example:

You create a project today.

Tomorrow, you add new files.

Next week, you fix a mistake.

Git can help you see what changed and when.

Simple meaning:

Git helps you track your work step by step.

18. What Is Version Control?

Version control is a system that records changes over time.

It helps developers:

- Save project history
- Compare changes
- Undo mistakes
- Work safely on new features
- Collaborate with others

- Understand who changed what

Simple meaning:

Version control is like a save-history system for code projects.

19. Why Developers Use Git

Developers use Git because it helps with:

- Tracking changes
- Saving project progress
- Recovering older versions
- Working in teams
- Reviewing changes
- Managing branches
- Collaborating through GitHub or GitLab

Simple meaning:

Git helps developers work safely and professionally.

20. What Is a Commit?

A **commit** is a saved snapshot of your project.

Simple meaning:

A commit is one saved point in your project history.

Example:

```
Commit 1: created homepage  
Commit 2: added CSS file  
Commit 3: fixed spelling mistakes
```

Each commit records changes at a specific time.

A commit can help you go back and understand what happened in your project.

Simple meaning:

A commit is like saving a clear checkpoint in your project.

21. What Is a Branch?

A **branch** is a separate path in your project history.

Branches let you work on changes without directly changing the main stable version.

Example:

```
main branch = stable project
feature branch = new work or experiment
```

Simple meaning:

A branch lets you work separately without damaging the main project.

You will learn branches more deeply in later sprints.

For Sprint 5, just understand that branches help isolate work.

22. Git vs GitHub

Many beginners confuse Git and GitHub.

They are related, but they are not the same.

Tool	Simple Meaning
Git	Tool on your computer that tracks project changes
GitHub	Website that stores Git projects online
GitLab	Another website that stores Git projects online

Simple meaning:

Git is the tracking tool. GitHub and GitLab are online hosting platforms.

23. What Are GitHub and GitLab?

GitHub and GitLab are cloud version control providers.

They can store Git repositories online.

They also provide collaboration tools such as:

- Browser code review
- Issue tracking
- Project boards
- Pull requests
- Suggested edits
- Merging tools
- Team collaboration

Simple meaning:

GitHub and GitLab are websites where developers store and collaborate on Git projects.

You will learn GitHub more deeply in later sprints.

24. What Is a Repository?

A repository is a project folder tracked by Git.

Simple meaning:

A repository is a Git-tracked project.

A repository can be:

- Local: stored on your computer
- Remote: stored online on GitHub or GitLab

You will learn repositories in Sprint 6 and Sprint 7.

For Sprint 5, remember:

Git tracks project history inside repositories.

25. Checking Whether Git Is Installed

To check if Git is installed, run:

```
git --version
```

Example output:

```
git version 2.45.0
```

If you see a version number, Git is installed.

Simple meaning:

```
git --version
```

 checks whether Git is available on your computer.

26. Installing Git

If Git is not installed, install it using the method for your operating system.

Ubuntu / Debian Linux

```
sudo apt-get install git
```

Simple meaning:

This installs Git on Ubuntu or Debian-based Linux systems.

Arch Linux

```
sudo pacman -S git
```

Simple meaning:

This installs Git on Arch Linux.

macOS with Homebrew

```
brew install git
```

Simple meaning:

This installs Git on macOS using Homebrew.

Windows with Chocolatey

```
choco install git.install
```

Simple meaning:

This installs Git on Windows using Chocolatey.

Windows Git Bash Note

On Windows, Git often comes with **Git Bash**.

Git Bash gives Windows users a Bash-like terminal.

Simple meaning:

Git Bash helps Windows users run Git and Bash-like commands.

27. Git Configuration

After installing Git, you should configure it.

Git configuration means setting Git options.

The most important first settings are:

- `user.name`
- `user.email`
- `core.editor`

Simple meaning:

Git configuration tells Git how to behave and who you are.

28. Why Git Needs Your Name and Email

Every Git commit records the author.

Example:

```
Author: Maleesha <maleesha@example.com>
```

Git gets this information from:

```
user.name  
user.email
```

Simple meaning:

Git uses your name and email to label your commits.

29. Setting Your Git Username

Use this command:

```
git config --global user.name "Your Name"
```

Example:

```
git config --global user.name "Maleesha"
```

If your name has spaces, keep it inside quotes.

Example:

```
git config --global user.name "Maleesha Perera"
```

Simple meaning:

This tells Git the name to attach to your commits.

30. Setting Your Git Email

Use this command:

```
git config --global user.email "your@email.com"
```

Example:

```
git config --global user.email "maleesha@example.com"
```

Simple meaning:

This tells Git the email to attach to your commits.

Beginner note:

If you plan to use GitHub, it is better to use the email connected to your GitHub account or your GitHub private email.

31. What Does `--global` Mean?

Look at this command:

```
git config --global user.name "Maleesha"
```

The part:

```
--global
```

means the setting applies to all Git projects on this computer.

Simple meaning:

Global means everywhere on this computer.

For beginners, using `--global` is usually correct when setting your Git name and email.

32. Global Config vs Project-Level Config

Git settings can be stored at different levels.

Global config

Global config applies to all Git projects for the current user.

Example:

```
git config --global user.name "Maleesha"
```

Simple meaning:

Global config is the default setting for all your projects.

Project-level config

Project-level config applies only to one specific repository.

You usually set this inside a project folder without `--global`.

Example idea:

```
git config user.name "Project Name"
```

Simple meaning:

Project-level config only affects the current Git project.

Beginner note:

In Sprint 5, use global config unless you have a special reason not to.

33. Setting the Preferred Git Editor

Sometimes Git needs to open a text editor.

Example:

Git may open an editor when writing a longer commit message.

You can tell Git to use VS Code.

Command:

```
git config --global core.editor "code --wait"
```

Simple meaning:

This tells Git to use VS Code when Git needs an editor.

34. What Does `core.editor` Mean?

`core.editor` is the Git setting for your preferred text editor.

Example:

```
core.editor=code --wait
```

This means Git will open VS Code when it needs an editor.

Simple meaning:

`core.editor` tells Git which editor to open.

35. Why `code --wait` Is Used

The command is:

```
git config --global core.editor "code --wait"
```

`code` opens VS Code.

`--wait` tells Git to wait until you close the VS Code editor window.

Simple meaning:

`code --wait` lets Git wait while you finish editing in VS Code.

36. Other Possible Git Editors

Git can use different editors.

Examples:

```
git config --global core.editor emacs
```

This sets Emacs as the Git editor.

Another example for Notepad++ on Windows:

```
git config --global core.editor '"C:/Program Files/Notepad++/notepad++.exe" -  
multiInst -notabbar -nosession -noPlugin'
```

Simple meaning:

Git can use many editors, but VS Code is beginner-friendly for this course.

37. Viewing Git Configuration

To see your Git configuration, run:

```
git config --list
```

Example output:

```
user.name=Maleesha  
user.email=maleesha@example.com  
core.editor=code --wait
```

Simple meaning:

`git config --list` shows your current Git settings.

38. Viewing Git Configuration with Origin Information

You can also see where each config value comes from.

Command:

```
git config --list --show-origin
```

Simple meaning:

This shows the config values and the files where they are stored.

This is useful when you want to know whether a setting came from global config or project config.

Part 3: Sprint 5 Commands and Practice

39. Sprint 5 Command Summary

Command / Shortcut	Simple Meaning
<code>nano <filename></code>	Open or create a file in Nano
<code>Ctrl + O</code>	Save file in Nano
<code>Enter</code>	Confirm file name after saving in Nano
<code>Ctrl + X</code>	Exit Nano
<code>git --version</code>	Check installed Git version
<code>sudo apt-get install git</code>	Install Git on Ubuntu/Debian Linux
<code>sudo pacman -S git</code>	Install Git on Arch Linux
<code>brew install git</code>	Install Git on macOS using Homebrew
<code>choco install git.install</code>	Install Git on Windows using Chocolatey
<code>git config --global user.name "Your Name"</code>	Set Git username globally
<code>git config --global user.email "your@email.com"</code>	Set Git email globally
<code>git config --global core.editor "code --wait"</code>	Set VS Code as Git editor
<code>git config --list</code>	Show Git config list
<code>git config --list --show-origin</code>	Show Git config list with source files

40. Full Sprint 5 Practice Flow

Complete this practice slowly and carefully.

Step 1: Open your terminal

Open the VS Code terminal or your system terminal.

Simple meaning:

You need a terminal to run Nano and Git commands.

Step 2: Create and open a file with Nano

```
nano sprint5.txt
```

Simple meaning:

Open a file called `sprint5.txt` in Nano.

Step 3: Type text inside Nano

Type:

```
I am learning Sprint 5.  
Nano helps me edit files in the terminal.
```

Simple meaning:

You are writing content inside the file.

Step 4: Save the file

Press:

```
Ctrl + O
```

Then press:

```
Enter
```

Simple meaning:

Save the file and confirm the name.

Step 5: Exit Nano

Press:


```
Ctrl + X
```

Simple meaning:

Close Nano and return to the terminal.

Step 6: Check the file content

```
cat sprint5.txt
```

Expected output:

```
I am learning Sprint 5.  
Nano helps me edit files in the terminal.
```

Simple meaning:

This confirms your file was saved correctly.

Step 7: Check Git version

```
git --version
```

Example output:

```
git version 2.45.0
```

Simple meaning:

This confirms Git is installed.

Step 8: Set your Git username

```
git config --global user.name "Your Name"
```

Example:

```
git config --global user.name "Maleesha"
```

Simple meaning:

Git will attach this name to your commits.

Step 9: Set your Git email

```
git config --global user.email "your@email.com"
```

Example:

```
git config --global user.email "maleesha@example.com"
```

Simple meaning:

Git will attach this email to your commits.

Step 10: Set VS Code as your Git editor

```
git config --global core.editor "code --wait"
```

Simple meaning:

Git will open VS Code when it needs an editor.

Step 11: View Git configuration

```
git config --list
```

You should see lines similar to:

```
user.name=Your Name  
user.email=your@email.com  
core.editor=code --wait
```

Simple meaning:

This shows that your Git setup is saved.

Part 4: Important Concepts in Simple Words

41. Why `user.name` Matters

`user.name` is the name Git writes on your commits.

Example:

```
Author: Maleesha
```

Simple meaning:

`user.name` tells Git who made the change.

42. Why `user.email` Matters

`user.email` is the email Git writes on your commits.

Example:

```
Author: Maleesha <maleesha@example.com>
```

Simple meaning:

`user.email` helps identify the commit author.

43. Good Checkpoint Answer

Question:

Why do `user.name` and `user.email` matter in Git?

Good beginner answer:

`user.name` and `user.email` matter because Git uses them to identify who made each commit.

Another good answer:

They are attached to commits, so other people can see who made the project change.

44. What Sprint 5 Is Not Yet About

Sprint 5 is not mainly about:

- Creating repositories
- Staging files
- Making commits
- Viewing commit logs
- Pushing to GitHub
- Creating branches deeply
- Pull requests

Those topics come later.

Sprint 5 is mainly about:

- Nano basics
- Git installation check
- Git setup
- Git identity configuration
- Git editor configuration

Simple meaning:

Sprint 5 prepares Git. Sprint 6 starts the local Git workflow.

Part 5: Common Beginner Mistakes

45. Mistake 1: Forgetting to Save in Nano

Problem:

You type text in Nano but forget to save it.

Better habit:

```
Ctrl + O  
Enter  
Ctrl + X
```

Simple meaning:

Save first, then exit.

46. Mistake 2: Pressing `Ctrl + X` and Ignoring the Save Question

If you press `Ctrl + X` without saving, Nano may ask if you want to save changes.

If you accidentally choose not to save, your changes may be lost.

Simple meaning:

When Nano asks to save changes, answer carefully.

47. Mistake 3: Confusing `^O` with the Letter O Only

In Nano:

```
^O
```

means:

```
Ctrl + O
```

It does not mean type the `^` symbol.

Simple meaning:

`^O` means hold Ctrl and press O.

48. Mistake 4: Thinking Git and GitHub Are the Same

Wrong idea:

```
Git = GitHub
```

Correct idea:

```
Git = tool on your computer  
GitHub = website for Git projects
```

Simple meaning:

Git tracks changes. GitHub stores Git projects online.

49. Mistake 5: Not Checking Whether Git Is Installed

Before using Git, always check:

```
git --version
```

Simple meaning:

Check Git before trying to use Git commands.

50. Mistake 6: Forgetting Quotes Around a Name with Spaces

Wrong:

```
git config --global user.name Maleesha Perera
```

Better:

```
git config --global user.name "Maleesha Perera"
```

Simple meaning:

Use quotes when the value has spaces.

51. Mistake 7: Writing the Wrong Email

Problem:

You type the wrong email in Git config.

Better:

Check carefully before pressing Enter.

Example:

```
git config --global user.email "your@email.com"
```

Simple meaning:

Your Git email should be correct because it appears in commit history.

52. Mistake 8: Forgetting `--global`

If you forget `--global`, Git may set the value only for one project or may require you to be inside a repository.

For Sprint 5, use:

```
git config --global user.name "Your Name"
```

and:

```
git config --global user.email "your@email.com"
```

Simple meaning:

Use `--global` for your first Git setup.

53. Mistake 9: Thinking Sprint 5 Means You Already Know Git Fully

Sprint 5 only introduces Git setup.

You will learn more Git in later sprints:

- Sprint 6: Local Git workflow
- Sprint 7: GitHub repository and remotes
- Sprint 8: Branches, pull requests, and code review
- Sprint 9: Collaboration, forks, `.gitignore`, Pages, conflicts
- Sprint 10: Industry Git expansion and CI/CD

Simple meaning:

Sprint 5 is the beginning of Git, not the whole Git topic.

Part 6: Key Terms Summary

54. Sprint 5 Key Terms

Term	Simple Meaning
Nano	A terminal text editor
Terminal editor	An editor that works inside the terminal
Vim	Powerful terminal editor, harder for beginners
Emacs	Powerful editor, harder for beginners
<code>^</code> in Nano	Means Ctrl
<code>M-</code> in Nano	Means Meta key, usually Alt
<code>Ctrl + O</code>	Save file in Nano
<code>Ctrl + X</code>	Exit Nano
Write Out	Nano word for saving a file
Git	Tool that tracks project changes
Version control	System that records project changes over time
Commit	Saved snapshot of project changes
Branch	Separate path for isolated work
GitHub	Online platform for Git repositories
GitLab	Another online platform for Git repositories
Repository	A project tracked by Git

Term	Simple Meaning
Local repository	Git project on your computer
Remote repository	Git project stored online
Git config	Settings that control Git behavior
<code>user.name</code>	Name attached to commits
<code>user.email</code>	Email attached to commits
<code>core.editor</code>	Editor Git opens when needed
<code>--global</code>	Setting applies to all projects for the current user
Git Bash	Bash-like terminal for Windows users

Part 7: Final Sprint 5 Understanding

55. What You Should Understand After Sprint 5

After Sprint 5, you should understand that Nano is a simple terminal editor.

You should be able to:

- Open a file using Nano.
- Type text inside Nano.
- Save using `Ctrl + O`.
- Confirm using `Enter`.
- Exit using `Ctrl + X`.
- Understand that `^` means Ctrl.
- Understand that `M-` usually means Alt.

You should also understand that Git is a version control system.

You should be able to:

- Explain what Git is.
- Explain what version control means.
- Explain what a commit is.
- Explain what a branch is.
- Explain Git vs GitHub.
- Check Git version using `git --version`.
- Install Git if needed.
- Set your Git username.

- Set your Git email.
- Set VS Code as the Git editor.
- View Git config using `git config --list`.

Simple final meaning:

Sprint 5 prepares your terminal editing skill and Git identity setup before you start using Git repositories in Sprint 6.

Part 8: Sprint 5 Practice Task

56. Practice Task: Nano and Git Setup

Complete this task step by step.

Task A: Nano Practice

Run:

```
nano sprint5.txt
```

Type:

```
I am learning Sprint 5.  
Nano helps me edit files in the terminal.
```

Save:

```
Ctrl + O  
Enter
```

Exit:

```
Ctrl + X
```

Check:

```
cat sprint5.txt
```

Expected output:

```
I am learning Sprint 5.  
Nano helps me edit files in the terminal.
```

Task B: Git Version Check

Run:

```
git --version
```

Expected idea:

```
git version x.x.x
```

Simple meaning:

This confirms Git is installed.

Task C: Git Configuration

Run these commands using your real name and email:

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"  
git config --global core.editor "code --wait"
```

Simple meaning:

This configures your Git identity and preferred editor.

Task D: Check Git Configuration

Run:

```
git config --list
```

Look for:

```
user.name=Your Name  
user.email=your@email.com  
core.editor=code --wait
```

Simple meaning:

This confirms your Git configuration is saved.

Part 9: Sprint 5 Checkpoint

57. Sprint 5 Checkpoint

Your checkpoint is:

1. Show `git --version` output.
2. Show `git config --list` output.
3. Explain why `user.name` and `user.email` matter.

Required commands:

```
git --version
```

```
git config --list
```

Good explanation:

```
user.name and user.email matter because Git uses them to identify who made each  
commit.
```

Simple meaning:

This checkpoint proves that Git is installed and configured correctly.

Part 10: Sprint 5 Mini Assessment

58. Answer These Questions Before Moving to Sprint 6

1. What is Nano?
2. Why is Nano useful?
3. How do you open a file using Nano?
4. What does `^` mean in Nano?
5. What does `M-` mean in Nano?
6. How do you save a file in Nano?
7. What does `Ctrl + O` do?
8. What should you press after `Ctrl + O` to confirm the file name?
9. How do you exit Nano?
10. What does `Ctrl + X` do?
11. What is Git?
12. What is version control?
13. Why do developers use Git?
14. What is a commit?
15. What is a branch?
16. What is GitHub?
17. What is GitLab?
18. What is the difference between Git and GitHub?
19. What command checks the Git version?
20. What command installs Git on Ubuntu/Debian?
21. What command installs Git on Arch Linux?
22. What command installs Git on macOS using Homebrew?
23. What command installs Git on Windows using Chocolatey?
24. What is Git configuration?
25. What does `user.name` mean?
26. What does `user.email` mean?
27. Why do `user.name` and `user.email` matter?
28. What does `--global` mean?
29. What does `core.editor` mean?
30. What command sets VS Code as the Git editor?
31. What command shows your Git configuration?
32. What command shows config values with their origin files?
33. Why should you use quotes around names with spaces?
34. Why is Sprint 5 only Git setup and not full Git workflow yet?

Part 11: Sprint 5 Completion Checklist

59. Tick These When Completed

- ☐ I can explain what Nano is.
 - ☐ I can open a file using `nano <filename>`.
 - ☐ I can type text inside Nano.
 - ☐ I understand that `^` means Ctrl in Nano.
 - ☐ I understand that `M-` usually means Alt in Nano.
 - ☐ I can save a file in Nano using `Ctrl + O`.
 - ☐ I can confirm the file name using `Enter`.
 - ☐ I can exit Nano using `Ctrl + X`.
 - ☐ I can check a saved file using `cat`.
 - ☐ I can explain what Git is.
 - ☐ I can explain what version control is.
 - ☐ I can explain what a commit is.
 - ☐ I can explain what a branch is.
 - ☐ I can explain Git vs GitHub.
 - ☐ I can check Git version using `git --version`.
 - ☐ I know the Git installation command for my operating system.
 - ☐ I can set Git username using `git config --global user.name`.
 - ☐ I can set Git email using `git config --global user.email`.
 - ☐ I can set VS Code as Git editor using `git config --global core.editor "code --wait"`.
 - ☐ I can view Git configuration using `git config --list`.
 - ☐ I can explain why `user.name` and `user.email` matter.
 - ☐ I completed the Sprint 5 checkpoint.
-

60. End of Sprint 5 Notes

You are ready to move to **Sprint 6: Local Git Workflow** only after you can complete the Sprint 5 checkpoint confidently.

Before moving forward, make sure you can run and explain these commands:

```
git --version
```

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

```
git config --global core.editor "code --wait"
```

```
git config --list
```

Final simple meaning:

Sprint 5 teaches you how to edit files in the terminal using Nano and how to configure Git correctly before starting real Git project work.

In Sprint 6, you will start using Git locally by learning:

- Repository
- Hidden `.git` folder
- Working tree
- File states
- Staging area
- Commits
- Commit messages
- Commit history
- `git log`

So Sprint 5 is the setup stage.

Sprint 6 is the first real Git workflow stage.